PARTE 3: Análise e Governança do Pipeline

1. Qual regra da Silver mandou mais registros para a quarentena e por quê?

A regra que mais mandou registos para a quarentena foi a de **integridade de chaves primárias e valores válidos** (campos como id_pedido ou id_cliente nulos/vazios, e valores de transação ou quantidades zerados/negativos).

Porquê? Os dados brutos da *Landing Zone* vêm de sistemas transacionais e eventos de cliques que frequentemente geram ruído (pedidos cancelados sem fecho, falhas de rede ou requisições incompletas). Como o contrato de dados na passagem da Bronze para a Silver exige identificadores preenchidos e valores positivos, qualquer linha quebrada é desviada automaticamente para a quarentena para não corromper os cálculos analíticos posteriores.

2. Se a reconciliação não fechasse, o que você investigaria primeiro?

A primeira coisa que eu faria era bater a matemática básica: **(Registos válidos na Silver) + (Registos na Quarentena) = (Total ingerido na Bronze)**.

Se a diferença coincidir exatamente com os dados retidos pelas regras de qualidade (DQ), o motivo fica imediatamente esclarecido. Se ainda assim faltar dado, eu investigaria em seguida:

- Se a lógica de deduplicação (dropDuplicates) descartou linhas legítimas por critérios de chave excessivamente agressivos;
- Se atualizações de estado de pedidos geraram duplicações em vez de substituições (*merge*) nas tabelas Iceberg;
- Se houve desfasamento de fuso horário (*timezone* UTC vs. local) fazendo com que vendas de final de noite fossem contabilizadas no dia seguinte.

3. Que dado pessoal existe na sua arquitetura e como ele foi tratado (mascarado, removido, restrito)?

A base contém dados de clientes (PII), especificamente **nome completo, CPF, e-mail e telefone**.

O tratamento foi aplicado da seguinte forma:

- **Mascaramento e Hash:** Na camada Silver, campos como CPF e e-mail sofrem anonimização ou cálculo de *hash* criptográfico para evitar exposição direta em texto limpo;
- **Acesso Restrito:** As camadas com os dados detalhados (Bronze e Silver) permanecem isoladas no armazenamento de objetos, acessíveis apenas pelo processo e pelo utilizador de administração;

- **Omissão total na camada de entrega:** Na camada Gold e no PostgreSQL (schema: gold), o modelo expõe apenas métricas consolidadas (faturação por categoria, médias de avaliação e volume por região). O utilizador de leitura do Metabase consome apenas dados agregados, sem acesso a dados pessoais.

4. O que você mudaria nesta arquitetura se o volume de dados fosse 1.000 vezes maior?

Com uma volumetria 1.000 vezes superior, o ambiente local e os contentores simples não comportariam o processamento. Eu faria as seguintes mudanças estruturais:

- **Processamento Distribuído em Nuvem:** Substituiria o PySpark local por um cluster gerido escalável horizontalmente (como Databricks, AWS EMR ou Spark em Kubernetes);
- **Storage Nativo de Nuvem:** Trocaria o MinIO local por um serviço de *object storage* de nuvem elástico (Amazon S3, Google Cloud Storage ou Azure Data Lake);
- **Particionamento e Otimização Iceberg:** Implementaria particionamento por datas (ano/mês/dia) e rotinas automáticas de manutenção no Iceberg (como rewrite_data_files para compactar pequenos ficheiros e expiração periódica de *snapshots*);
- **Serving Layer Dedicada a OLAP:** Substituiria o PostgreSQL de nó único por um motor analítico colunar (como ClickHouse, Snowflake, BigQuery ou Trino a consultar diretamente as tabelas Gold em Iceberg) para suportar o volume de consultas do Metabase sem degradação de desempenho.